

5.6 *Pokazivači u C++*

5.6.1 Osnove pokazivača

5.6.2 Osnovna područja primjene pokazivača

5.6 Pokazivači u C++

Pokazivači u osnovi predstavljaju memorijske adrese. Na tim adresama mogu se nalaziti podaci različitih tipova : varijable, funkcije ili objekti. Takve podatke kojima pristupamo preko pokazivača nazivamo dinamičkim, i to zato jer njihova lokacija u memoriji nije zadana prilikom prevođenja programa, nego tek u toku izvršavanja.

Upotreba pokazivača doprinosi fleksibilnosti programiranja : pokazivačka varijabla koja sadrži adresu jednog dinamičkog podatka može se “preusmjeriti” tako da pokazuje na neki drugi dinamički podatak istog tipa.

5.6.1 Osnove pokazivača

Uobičajena, tzv. statička varijabla predstavlja imenovani memorijski prostor, koji sadrži vrijednost. Na primjer, ako napišemo ovako :

```
int a;  
a = 10;
```

time određujemo da se u toku prevođenja programa rezervira memorijski prostor veličine 4 bajta, kojemu ćemo pristupati preko identifikatora *a*. U toku izvođenja programa možemo tom prostoru mijenjati vrijednost, ali mu ne možemo mijenjati memorijsku lokaciju.

Za razliku od statičke varijable, pokazivač je varijabla koja pokazuje na drugu varijablu. To znači da ona sadrži memorijsku adresu na kojoj se nalazi vrijednost. Taj memorijski sadržaj, na koji pokazuje pokazivač, predstavlja *dinamičku varijablu*.

Pokazivače označavamo znakom *, odnosno :

```
int *p;
```

ovime kažemo da je varijabla *p* pokazivač na dinamičku varijablu tipa *int*. Sada pokazivačkoj varijabli treba dodijeliti memorijski prostor za dinamičku varijablu :

```
p = new int;
```

time osiguramo da neka nova naredba *new* ne zauzme isti memorijski prostor, odnosno, da ne “brljamo” po već rezerviranom prostoru. Dinamičkoj varijabli možemo pridružiti vrijednost :

```
*p = 100;
```

U slijedećem primjeru treba obratiti pažnju na razliku između pridruživanja vrijednosti dinamičkim varijablama i pridruživanja vrijednosti pokazivačima :

```
int *a = new int, *b = new int;  
*a = 5;  
*b = *a;      // pridruživanje vrijednosti dinamičkoj varijabli  
b = a;        // pridruživanje vrijednosti pokazivaču, odnosno, sada  
              // oba pokazivača sadrže adresu iste dinamičke varijable  
*b = 10;      // tako da ova instrukcija dovodi do toga  
cout << *a;   // da i *a ima vrijednost 10
```

Kada dinamička varijabla više nije potrebna, poželjno je osloboditi (deallocirati) memorijski prostor koji zauzima :

```
delete p;
```

Time se memorijski prostor dinamičke varijable oslobađa za neku drugu namjenu, ali pokazivač zadržava staru vrijednost, što je potencijalno opasno. Zbog toga je poželjno takvom, trenutno slobodnom pokazivaču pridružiti vrijednost NULL (nulta adresa, označava neupotrijebljeni pokazivač) :

```
p = NULL;  
ili  
p = 0;
```

Izrazi su ekvivalentni, jer je NULL definiran kao konstanta s vrijednošću 0.

5.6.1.1 Korištenje adresnog operatora (&)

Dok pokazivači sadrže adrese dinamičkih podataka, pomoću adresnog operatora možemo dohvatiti adresu nekog statičkog podatka. Primjer :

```
int *pok;  
int a = 10;  
pok = &a;           // usmjeravamo pokazivač na statičku varijablu a  
cout << *pok << endl; // ispisuje se 10
```

Izraz *&a* ima značenje “memorijska adresa varijable *a*” i može se pridružiti pokazivaču. Adresni operator možemo iskoristiti za prosljeđivanje adrese neke podatkovne strukture funkciji :

```
#include <iostream.h>  
#include <string.h>  
struct tslog{  
    int mat_br;  
    char prez_i_ime[30];  
};  
void ispis(tslog *arg){           // pokazivač arg dobiva adresu strukture  
    cout << arg -> mat_br << endl;  
    cout << arg -> prez_i_ime << endl;  
};  
void main(){  
    tslog slog;  
    slog.mat_br = 32000;  
    strcpy (slog.prez_i_ime, "Matić Mate");  
    ispis (&slog); // prosljeđujemo adresu strukture  
}
```

U primjeru prosljeđujemo funkciji *ispis* adresu strukture *slog*, koju ova dohvaća pomoću pokazivača **arg*.

5.6.1.1.1 Reference

Reference se označavaju adresnim operatorom (&), a služe kao drugo ime za neki memorijski prostor. Primjer :

```
int a = 10;
```

```
int &b = a;
cout << b << endl;    // 10
```

Identifikator *b* zapravo je drugo ime za varijablu *a*, odnosno, predstavlja isti memorijski prostor.

Posebno je interesantna mogućnost da se referenca iskoristi u listi argumenata funkcije. U tom slučaju gornji primjer izgledao bi ovako:

```
#include <iostream.h>
#include <string.h>
struct tslog{
    int mat_br;
    char prez_i_ime[30];
};
void ispis(tslog &arg){                // referenca arg predstavlja drugo ime za
                                     // strukturu slog
    cout << arg.mat_br << endl;
    cout << arg.prez_i_ime << endl;
};
void main(){
    tslog slog;
    slog.mat_br = 32000;
    strcpy (slog.prez_i_ime,"Matić Mate");
    ispis (slog);    // identifikator slog bit će zamijenjen referencom
}
```

U ovom slučaju argument *arg* predstavlja drugo ime za memorijski prostor koji zauzima struktura *slog*.

5.6.2 Osnovna područja primjene pokazivača

5.6.2.1 Kreiranje dinamičkih struktura podataka

Strukture podataka koje zauzimaju memorijski prostor u toku izvođenja programa nazivamo dinamičkim. Tipični primjer za dinamičke strukture podataka je vezana lista. Svaki element vezane liste sadrži pokazivač s adresom slijedećeg (eventualno i drugi pokazivač s adresom prethodnog) elementa liste. Pokazivač zadnjeg elementa u listi ima vrijednost *NULL*, što označava kraj liste. U slučaju da dodajemo novi element u listu, tada jednostavno njegovu adresu pridružimo pokazivaču do sada zadnjeg elementa liste. Dakle, broj elemenata nije unaprijed zadan, nego se mogu dodavati novi tako dugo dok ima raspoloživog memorijskog prostora.

Vezana lista je vrlo pogodna struktura za objektno-orijentirano programiranje, kakvo nam omogućuje C++. Naime, elementi liste mogu biti objekti, implementacije jedne, zajedničke osnovne klase ili iz njih izvedenih klasa. Iz toga proizlazi da elementi liste mogu biti heterogeni. Zahvaljujući polimorfizmu, svim elementima liste možemo pristupiti pomoću istog pokazivača (vidi : Primjer (dvostruko vezana lista s heterogenim elementima)).

5.6.2.2 Pokazivači i reference kao zamjena za varijabilne argumente funkcija

Varijabilni argumenti potprograma mogući su u Pascalu. Oznaka *var* u listi formalnih argumenata znači da će se ta vrijednost vratiti u pozivajući potprogram, odnosno, stvarni parametar poprimiti će vrijednost formalnog. Primjer (Pascal) :

```
var x, y : integer;
procedure proc1 (a : integer; var b : integer);
begin
    a := 10;
    b := 10;
end;
begin
    x := 5;
    y := 5;
    proc1 (x,y);
    writeln ('x = ', x);
    writeln ('y = ', y);
end;
```

Ispisuje se :

```
x = 5
y = 10
```

Varijabla *y* i varijabilni parametar *b* predstavljaju istu memorijsku lokaciju, pa je tako procedura vratila vrijednost u glavni program.

Odgovarajući primjer može se riješiti jeziku u C++ korištenjem pokazivača i referenci :

a.) na način C-a :

```
#include <stdio.h>
int x, y;
void proc1 (int a, int *b){
    a = 10;
    *b = 10;
};
void main (){
    x = 5;
    y = 5;
    proc1 (x, &y);
    printf ("x = %d\n",x);
    printf ("y = %d\n",y);
}
```

U listi stvarnih argumenata koristimo referencu (&) na varijablu *y*, odnosno, umjesto vrijednosti, prosjeđujemo adresu varijable *y*. Toj adresi pristupamo pomoću pokazivača *b* (*b = 10) koji je definiran u listi formalnih argumenata.

b.) na način C++ :

```
#include <iostream.h>
int x, y;
void proc1 (int a, int &b){
    a = 10;
```

```

        b = 10;
    };
    void main () {
        x = 5;
        y = 5;
        proc1 (x, y);
        cout << "x = " << x << endl;
        cout << "y = " << y << endl;
    }

```

C++ omogućuje korištenje referenci u listi formalnih argumenata funkcije, pa je rješenje još jednostavnije : umjesto vrijednosti, parametar *b* ima istu adresu kao i varijabla *y*.

5.6.2.3 Rad sa znakovnim nizovima

5.6.2.3.1 Polja znakova

Niz znakova (za koji u Pascalu koristimo tip *string*) u C/C++ predstavlja jednostavno polje znakova. U slijedećoj programskoj liniji definiramo polje znakova koje može sadržavati do 100 znakova :

```
char niz[101];
```

U ovom polju *niz[0]* predstavlja prvi znak niza, a *niz[100]* zadnji znak u nizu. Na kraju niza nalazi se znak za terminiranje niza – ASCII vrijednost nula (nul znak), koji se označava s *\0*. Odnosno, kažemo da C/C++ radi s nul-terminiranim nizovima znakova. Primjer :

```
char niz[101] = "Niz znakova";
```

Vrijednosti pojedinih elemenata niza su slijedeće :

```
niz[0] = 'N' . . . niz[10] = 'a', niz[11] = '\0'
```

Zbog toga kod definiranja znakovnih nizova moramo uvijek predvidjeti mjesto za znak za terminiranje niza.

Takav način rada sa znakovnim nizovima razlikuje se od načina na koji su nizovi znakova implementirani u Pascalu. Primjer (Pascal) :

```
var
    niz : string [100];
```

Definiran je niz znakova veličine najviše 100 znakova. Međutim, ne koristi se znak za terminiranje, nego nulti član niza sadrži podatak o dužini niza. Primjer :

```
niz := 'Niz znakova';
```

Vrijednosti pojedinih elemenata niza su slijedeće :

```
niz[0] = chr(11) . . . niz[1] = 'N', niz[11] = 'a'
```

Iz ovoga se također vidi da nizovi u Pascalu ne mogu sadržavati više od 255 znakova, dok u C/C++ nisu ograničeni po dužini.

5.6.2.3.2 Operacije nad poljima znakova

Za razliku od npr. Pascala, C/C++ nema ugrađenu podršku za rad s nizovima znakova, nego se oslanja na standardne biblioteke. Na primjer, želimo li u C++ realizirati nešto ovakvo (primjer u Pascalu) :

```
var
    niz1, niz2 : string;
begin
    niz1 := "Primjer u Pascalu";
    niz2 := niz1;
    if niz1 = niz2 then
        writeln ('Nizovi su isti!');
end;
```

morali bi se osloniti na funkcije definirane u biblioteci *string* :

```
#include <iostream.h>
#include <string.h>
void main(){
    char niz1[100], niz2[100];
    strcpy (niz1, "Primjer u C/C++");
    strcpy (niz2, niz1);
    if (strcmp(niz1, niz2) == 0)
        cout << "Nizovi su isti!" << endl;
}
```

U C/C++ ne može se jednostavno pridružiti jedno polje drugom. Umjesto toga za nizove znakova koristimo funkciju *strcpy*, odnosno funkciju *strcmp* za uspoređivanje vrijednosti nizova. Za slučaj da želimo spojiti nizove, odnosno saznati duljinu niza, koristimo funkcije *strcat*, odnosno *strlen* :

```
strcpy (niz1, "Primjer");
strcpy (niz2, " u C/C++");
strcat (niz1,niz2);
cout << niz1 << endl; // ispisuje se : Primjer u C/C++
cout << "Duljina niza : " << strlen(niz1) << endl;    // 15
```

5.6.2.3.3 Pokazivači i znakovni nizovi

Pokazivači mogu pojednostaviti rad sa znakovnim nizovima. Primjer :

```
#include<iostream.h>
void main(){
    char *s2;
    s2 = "Programiranje u C++";
    cout << s2 << endl;
}
```

Ispisuje se :

Programiranje u C++

Instrukcija `cout` u ovom slučaju ne ispisuje adresu, nego, zahvaljujući ugrađenoj konverziji tipova podataka (`s2` je definiran kao pokazivač na znakovni tip podataka), sadržaj odgovarajućih memorijskih lokacija, do znaka za terminiranje. U slučaju da želimo ispisati adresu, tada koristimo odgovarajući operator dodjele tipa :

```
cout << (int)s2 << endl;    // ispisuje se adresa
```

U slijedećem primjeru usmjeravamo pokazivač na polje znakova :

```
#include <string.h>
#include <iostream.h>
void main(){
    char s1[100];           // polje od 100 znakova
    strcpy (s1,"Programiranje u C++");

    char *s2=&s1[0];        // pokazivač na početni znak u polju
    cout << s2 << endl;
    s2 += 16;               // povećavamo adresu za 16
    cout << s2 << endl;
}
```

Ispisuje se :

```
Programiranje u C++
C++
```

U prvom primjeru usmjeravamo pokazivač `s2` na početni znak u nizu i ispisujemo sadržaj memorijskih lokacija između adrese `s2` i znaka za terminiranje. Kod drugog ispisa krećemo s ispisom od 17. znaka u nizu.

U slučaju da stavimo :

```
cout << *s2 << endl;       // ispisuje se 'C'
```

Ispisujemo sadržaj memorijske lokacije na koju pokazuje pokazivač `s2`. Obzirom na to da je `s2` definiran kao pokazivač na podatak tipa `char`, ispisat će se samo jedan znak.

U slučaju da želimo definirati dvodimenzionalno polje znakova, možemo to učiniti kao u slijedećem primjeru :

```
#include <iostream.h>
void main(){
    char ispiti [3][16] = {"Matematika","Statistika","Programiranje I"};
    cout << ispiti[2] << endl;    // Programiranje I
}
```

Ovaj pristup ima taj nedostatak da moramo zadati obje dimenzije niza, što znači da ćemo imati neiskorišteni memorijski prostor. Umjesto toga, možemo definirati polje pokazivača :

```
#include <iostream.h>
void main(){
    char *ispiti[4] = {"Matematika","Statistika","Programiranje I"};
    cout << ispiti[2] << endl;    // Programiranje I
    ispiti[3] = "Engleski jezik";
    cout << ispiti[3] << endl;    // Engleski jezik
}
```

Sadržaj odgovarajućih memorijskih lokacija bit će slijedeći (u uglatoj zagradi je indeks polja pokazivača *ispiti*) :

[0]											[1]									
M	a	t	e	m	a	t	i	k	a	\0	S	t	a	t	i	s	t	i	k	a

[2]																[3]				
\0	P	r	o	g	r	a	m	i	r	a	n	j	e		l	\0	E	n	g	l

e	s	k	i		j	e	z	i	k	\0										

Dakle, vidimo da ovdje imamo bolje iskorištenje memorijskog prostora. Ako gornjem promjeru dodamo slijedećih nekoliko programskih redaka :

```
char *pom;
pom = ispiti[0];
ispiti[0] = "Oblikovanje baza podataka";
cout << ispiti[0] << endl;      // Oblikovanje baza podataka
cout << pom << endl;          // Matematika
```

rezultat će biti da ćemo pokazivaču *ispiti[0]* pridružiti novi memorijski sadržaj (adresu teksta "Oblikovanje baza podataka"). Primjer radi, međutim, ostaje nam memorijski sadržaj na memorijskoj lokaciji na koju je prije pokazivao *ispiti[0]*, koju smo sačuvali u pokazivaču *pom*, na što treba obratiti pažnju u radu s pokazivačima, jer se prijašnja ušteda memorijskog prostora u tom slučaju gubi.